



Lovable Challenge @ Google

Solutions

“lv”-able

By: Joshua Andersson

Implemented by: Harry Zhang

Problem

We are given a string. We want to perform fewest number of the following operations such that it contains the substring "lv"

- Remove a character
- Insert any character anywhere
- Replace any character
- Reverse an interval

Insight

Answer is at most 2. Thus, it suffices to check if it's 0 or 1, otherwise we know it's 2. We only need the insert operation in an optimal solution

Only 0 if "lv" already present

Only 1 if "l" or "v" present. We can simply use the insert operation after it

Solution

```
n = int(input())
s = input()

if "lv" in s:
    print(0)
elif "l" in s or "v" in s:
    print(1)
else:
    print(2)
```

Challenge: can you solve the problem for a longer needle than "lv" in $O(\text{poly}(\text{needle length}))$?

Thirsty Cow

By: Joshua Andersson

Implemented by: Harry Zhang

Problem

We have a body of water. We know the water height it will have each of the following N days. If we drink at height x , we will drink until it has height 0. It will then decrease the height by x all coming days. We can only drink if the water height is positive. Maximize water consumption

Insight

It's never bad to drink water- it's as good to drink now as to wait and drink later.

Solution

$O(N^2)$ - simulate the greedy

$O(N)$ - optimize it

```
#!/usr/bin/python3
n = int(input())
a = [*map(int,input().split())]

out = 0
shift = 0
for x in a:
    if x-shift > 0:
        out += x-shift
        shift += x-shift

print(out)
```

Solution

Turns out, the answer is always the largest number in the input

```
input()
print(max(map(int,input().split())))
```

Unlockable

By: Harry Zhang

Problem

Given N integers $b_1, b_2, \dots, b_N$, as well as some integer $a > 1$.

Find total number of pairs (x, y) that satisfies $ya^x = b_i$,
for some i .

Insight

If we look at the integers x and y in the equation $ya^x = b_i$, we see that x and y cannot be that much larger than the largest b_i value.

This implies that we can try all possible x and y from 1 to 100, which is sufficient for the first subtask, with 22 points.

Insight

To continue on a similar idea, we note that since a is at least 2, x cannot be that large either.

In fact, if x is larger than 32, then $a^{32} > 10^9$.

Solution

To complete the solution, we can now iterate x over 1 to 32, and let try to find solutions for y . This can be done by looping over all unique b_i values, can check if a^x/b_i is an integer.

Since there are N values b_i , this algorithm runs in $O(32*N)$, and solves the problem.

Pitfall

Small pitfall:

Since a^x could be extremely large, we will either break early to not continue to calculate too large integers, or use 64- or 128-bit integers (if you do, watch out for if a is a power of 2, as it will overflow to 0, resulting in mod by 0).

Encryption

By: Joshua Andersson

Problem

A string is a subsequence of another if it can be obtained by deleting some characters while maintaining order

acb is a subsequence of f**a**jb**c**b

We are given a,b and the longest string that is a subsequence of both a and b

Find the shortest string that contains both a and b as subsequences

Pitfall

Solving subtasks 1-2, and not using
the given longest common subsequence

Problem is “impossible” without it



Subtask 1

$DP[i][j]$ = shortest string

$DP[i][j] = \max(1+DP[i-1][j], 1+DP[i][j-1],)$

if $a[i] == b[j]$:

$DP[i][j] = \max(DP[i][j], 1+DP[i-1][j-1])$

Subtask 2

Can be solved in $O(AB/64)$ using bitsets

Still too much memory - use Hirschberg's algorithm.

I hope no one did this

What we used to generate test data

Subtask 3

Longest common subsequence (LCS) and shortest common supersequence (SCS) are basically the same problem

We can greedily build the solution. If we think about it, the DP's for LCS and SCS are basically the same- but every character in LCS represents a point where we save a character in SCS.

Subtask 3

Iterate over the LCS, current char c , and keep an index i, j for each string.

While $a[i] \neq c$, increment i and add $a[i]$ to ans

While $b[j] \neq c$, increment j and add $b[j]$ to ans

Then, add c to ans, increment i, j and advance to next LCS char

Pitfalls

Printing 10^6 times in Python

Accidentally quadratic (insert etc)

Rotating Mugs

By: Harry Zhang

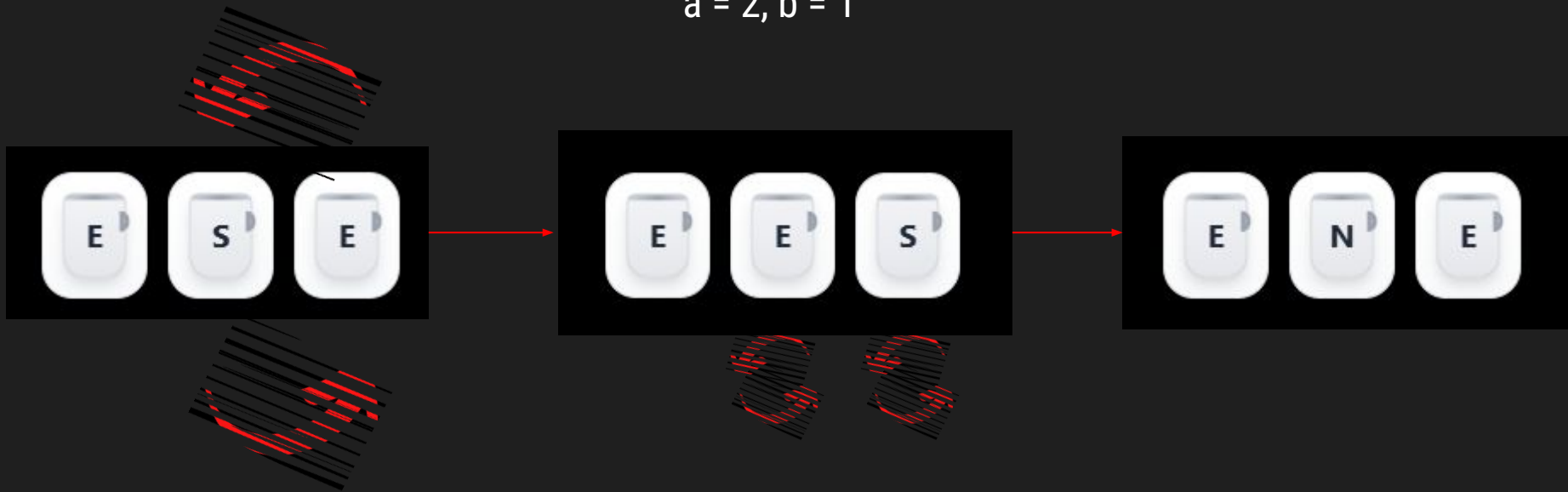
Problem

You are given M mugs that has 4 sides labelled “North”, “West”, “South” and “East”.

In each move (called Magical Rotations), we choose two mugs, switch their positions, and rotate one of those mugs as well as the mug to the right of it 90 degrees.

Find minimum number of moves such that all mugs are labelled “North”.

$a = 2, b = 1$



$a = 0, b = 1$



Insight

First insight is that the order of the mugs can be totally ignored, implying that we can just think of a Magical Rotation just as choosing any two indices i, j and rotate those two mugs by 90 degrees.

Proof/construction:

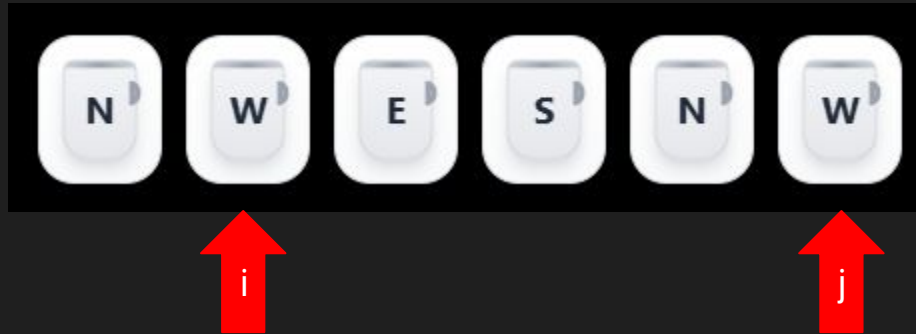
Let's say we want to rotate i and j where $i > 0$. Then we could just perform

Insight

Proof/construction:

Let's say we want to rotate i and j , and $i < j$.

When $i > 0$, then we could just preform the magical rotation where $a = j$, $b = i-1$



For $i = 0$, we can just let $a = 0$, and $b = j-1$

Insight

Now we can simply think of the mugs as the number of each sort of mug.

In other words, just keep track of number of North, West, South, and East we have. Each Magical rotation can be used to take any two mugs and move them to the “next” type.

Solution

25p - Only Norths and Easts initially.

Assign each face an integer, the minimum number of rotations it needs before it becomes North. So North - 0, West - 3, South - 2, East - 1. Consider the sum of this number for each of the mugs.

The parity of this sum is invariant when doing magical rotations. Implying that if the sum is negative, the answer is -1.

Solution

25p - Only Norths and Easts initially.

Otherwise, it should be possible. In fact, let E be the number of easts we have, which has to be even because of parity. The lower bound of number of moves is $E/2$, and this is achievable by repeatedly pairing easts.

Thus, a “greedy” solution works, and can solve this subtask.

Solution

$$35p - M \leq 100$$

Since the order doesn't matter, the state can be described with 4 integers, (N, W, S, E) , number of mugs of each label.

For a given M , how many different (N, W, S, E) are possible?

We can argue that it is bounded by $O(M^3)$, since if N, W, S are chosen as some integer between 1 and M , the last one, E has to make sure all integers sum up to M .

In other words, $E = M - (N + W + S)$.

Solution

$$35p - M \leq 100$$

This implies that we can run a BFS through all possible states (N, W, S, E) , until we reach $(M, 0, 0, 0)$. There are up to 4^2 possible states that we can reach from each state, since we can choose any two labels and transform them into the “next” one.

This results in $O(16 * M^3) = O(M^3)$, which is fast enough for this subtask.

Solution

Full solution

One could do a really greedy algorithm, by considering all tedious cases. An idea is to always pair up mugs of the same label. However, this specific greedy does not always work, as one could see from the samples:

ESE

Here we would want to rotate one of the E with the S, resulting in NEE, and then flip the last two Es together.

Solution

But if we have a lot of mugs of some label, one could show that no matter what the labels are, it is always optimal to pair it up with itself.

This implies that all mugs but a small amount of each label can be greedily paired up. For the remaining mugs that we have a small amount left of, we can use our solution from subtask 2.

This gives a full solution.

The ruler of everything

By: Joshua Andersson

Problem

You have 0 subscribers, and you want K . There are N possible videos you could make. If you have x subscribers before making a video, you have $a \cdot x + b$ afterwards, where a and b depend on the video. What is the smallest number of videos you need release? You can release them in any order, and at most once each.

Also, the ruler of everything is a good song



Problem

No greedy we are aware of works. We have to do DP or bruteforce

Subtask 1

Try all $N!$ orders of publishing videos. For each order, compute the smallest prefix needed.

Subtask 2

2^N bitmask DP

DP[mask]=max subscribers

No need to care about number of videos - that's encoded by mask

Subtask 2

Also possible to get $O(2^N \cdot N)$ by doing a DP, keeping track of how many of each a is used. Worst-case turns out to be all unique a .

Subtask 3

Problem so far: optimal order is difficult. Turns out, the optimal order can be found via sorting!

Consider two videos (a_1, b_1) and (a_2, b_2) . Is it better apply (a_1, b_1) or (a_2, b_2) first?

$$(a_1x + b_1)a_2 + b_2 = a_1a_2x + b_1a_2 + b_2$$

$$(a_2x + b_2)a_1 + b_1 = a_1a_2x + b_2a_1 + b_1$$

We can simply compare $b_1a_2 + b_2 > b_2a_1 + b_1$

Let's sort by this comparator

```
sort(all(vids), [](p2 a, p2 b)
{
    return (a.second*b.first)+b.second > (b.second*a.first)+a.second;
});
```

Subtask 3

Now that order is no issue, we can do DP to select which videos to publish.

$DP[i][j]$ = max subscribers if we have considered first i videos and published j of them

Knapsack-like:

$$DP[i][j] = \max(DP[i-1][j], DP[i-1][j-1] * a[i] + b[i])$$

To avoid overflow, do

$$DP[i][j] = \min(DP[i][j], K)$$

$$O(N^2)$$

Subtask 4

Need to reduce state. The “classical” approach of alien dp/ lagrange relaxation does not work as far as we know. Let us know if you manage to make it work!

Insight: we only ever use at most $\log(8e9)/\log(2)=33$ videos with $a > 1$.

Let's call videos with $a=1$ light and $a>1$ heavy.

Subtask 4

We first need to select some amount of light videos, call it L .

Obviously, we take the L videos with biggest b .

After making this choice, we need to compute the least number of heavy videos needed.

Let $f(L)$ = min number of heavy videos needed to get $\geq K$ subscribers if we take L light videos. Inf if impossible

Since $f(L) \leq 33$, we can compute it in $O(33N)$ using DP
(Simply disregard states with more than 33 videos)

Subtask 4

We now need to find the minima of $L+f(L)$. $O(N^2*33)$ if we try each L .

Instead, find first L such that $f(L) \neq \inf$ with binary search.

Suppose its value is s

We only need to check the 33 numbers in $[s, s+33]$, as any bigger value is $> s+f(s)$. Follows from $f(x) \leq 33$

$$O(33N*(\log(N)+33))$$

Subtask 5

If there are more than 33 heavy videos, the answer is at most 34. (Simply take all those videos)

Do DP with ALL videos, including those with $a=1$. Because we need at most 34, runs in $O(34N)$

We can look in last column of DP table to answer each query in $O(34)$

This is super fast, so from now on, we only consider the case where we have ≤ 33 heavy videos.

Subtask 5

Otherwise, there less than 34 heavy videos. $f(L)$ can be computed in $O(33^2)$

The smallest L can now be big, so we need to binary search for L . Each query can be answered in $O((33+\log(N))33^2) \sim 5 \cdot 10^4$
Fast enough for subtask 5

Subtask 6

Optimize the case with ≤ 33 heavy videos. Precompute each $f(L)$. $O(33^2 N)$. Now, queries can be answered in $O((33 + \log(N)) * 33)$.

Fast enough

Queries can be answered in $O(\log(N))$ or $O(\log(\max_K))$. $O(1)$ offline via radix sort

Subtask 6

Can also solve the problem using slope trick, ~300 LOC.

Outline:

Let $g(s, j)$ = max subscribers possible if we use j videos with $a > 2$, and start with s subscribers.

This function is *slope-trickable*

We can bound number of breakpoints to $\sqrt{\max_b}$ if we prune lines we can never use.

Tricky: to compute all $g(s, j)$, we need DP where we take max of two, and apply affine transformations. Doable, but hard to get right.

Subtask 6

After we have all $g(s,j)$, we can preprocess them to be computed in $O(\log(B))$ time

Infinity Alarm

By: Nils Gustafsson

Problem

Define $f(S)$ to be the set of integers x such that there is an odd number of elements in S that are strictly smaller than x .

Given N integers in the S , we let transform the set $S = f(S)$ once every second.

How many seconds does it take until S has an infinite amount of elements?

Harry promised to only talk about this problem if someone managed to solve it during contest.